

PROGRAMAÇÃO FRONT-END

ROTEIRO DE ATIVIDADE PRÁTICA

Galeria responsiva e otimizada com HTML e CSS

Curso	Técnico em Desenvolvimento de Sistemas
Público-alvo	Estudantes do Ensino Médio Técnico
Carga horária	3 aulas de 50 minutos
Unidade	Estilização de páginas, ferramentas e performance
Professor(a)	_____

Material adaptado a partir dos códigos de aula [SIS]ANO2C3B3S15A1, A2 e A3.

Nome: _____	Turma: _____
Data: ____/____/____	Dupla: _____

Situação-problema

Uma equipe desenvolveu uma galeria de projetos para o site da escola. A página funciona, mas apresenta três problemas: o layout não se adapta bem ao celular, todas as imagens são carregadas imediatamente e o arquivo CSS pode ser reduzido antes da publicação. Sua dupla deverá organizar e otimizar essa página.

Desafio

Criar uma galeria responsiva com CSS Grid, aplicar Lazy Loading nas imagens, gerar uma versão minificada do CSS e analisar a página com o Google Lighthouse.

Objetivos

- Construir uma página com estrutura semântica em HTML.
- Organizar cartões em um layout responsivo com CSS Grid.
- Aplicar o atributo loading="lazy" e textos alternativos nas imagens.
- Comparar o tamanho do CSS original com o CSS minificado.
- Executar uma auditoria de performance e registrar recomendações.

Tempo e organização

Duração	Forma de trabalho	Entrega
50 minutos	Duplas ou individual	Pasta do projeto + registro das respostas

Preparação necessária

O professor fornecerá três imagens chamadas imagem1.jpg, imagem2.jpg e imagem3.jpg. Caso os arquivos iniciais não sejam fornecidos, a digitação completa pode exigir uma segunda aula.

Materiais

- Computador com Visual Studio Code ou outro editor de texto.
- Navegador Google Chrome.
- Python instalado para o servidor local e a minificação adaptada.
- Três imagens fornecidas pelo professor.
- Pasta para salvar os arquivos do projeto.

Cronograma sugerido

Tempo	Etapas	Condução didática
0-5 min	Organização	Crie a pasta e confira as imagens.
5-17 min	HTML	Digite ou complete o arquivo index.html.
17-29 min	CSS Grid	Crie estilos.css e teste 3, 2 e 1 colunas.
29-37 min	Minificação	Execute o script e compare os tamanhos.
37-45 min	Auditoria	Abra a página no servidor e rode o Lighthouse.

Tempo	Etapas	Condução didática
45-50 min	Registro	Responda às perguntas e organize a entrega.

Etapa 1 - Criar a pasta e os arquivos

Crie uma pasta chamada galeria_otimizada. Dentro dela, organize os arquivos da seguinte maneira:

```
galeria_otimizada/
├── index.html
├── estilos.css
├── minificar_css.py
├── imagem1.jpg
├── imagem2.jpg
└── imagem3.jpg
```

Atenção

Os nomes dos arquivos devem ser exatamente iguais aos usados no código. Em alguns sistemas, imagem1.JPG e imagem1.jpg podem ser tratados como nomes diferentes.

Etapa 2 - Criar o HTML

Crie o arquivo index.html. O HTML organiza o conteúdo da página; o CSS será responsável pelo layout e pela aparência.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Galeria Responsiva</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <header class="cabecalho">
    <h1>Galeria de Projetos</h1>
    <p>Trabalhos desenvolvidos pela turma.</p>
  </header>

  <main class="galeria">
    <figure class="cartao">
      
      <figcaption>Projeto 1</figcaption>
    </figure>

    <figure class="cartao">
      
      <figcaption>Projeto 2</figcaption>
    </figure>

    <figure class="cartao">
      
      <figcaption>Projeto 3</figcaption>
    </figure>
  </main>
</body>
</html>
```

- Troque cada texto do atributo alt por uma descrição real da imagem.

- O atributo `loading="lazy"` adia o carregamento de imagens fora da área visível.
- `width` e `height` ajudam o navegador a reservar o espaço da imagem.
- A meta viewport permite que o layout responda corretamente em dispositivos móveis.

Verificação rápida do HTML

Item	Conferência
☐	O arquivo se chama <code>index.html</code> .
☐	O link aponta para <code>estilos.css</code> .
☐	Existem três elementos <code>figure</code> .
☐	Todas as imagens possuem <code>alt</code> , <code>width</code> , <code>height</code> e <code>loading</code> .
☐	As tags foram fechadas corretamente.

Etapa 3 - Criar o CSS responsivo

Crie o arquivo `estilos.css`. Durante o desenvolvimento, mantenha o código organizado e indentado para facilitar a leitura e a correção.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6f8;
  color: #1f2937;
}

.cabecalho {
  padding: 2rem 1rem;
  text-align: center;
}

.galeria {
  width: min(1100px, 92%);
  margin: 0 auto 3rem;
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}

.cartao {
  margin: 0;
  overflow: hidden;
  border-radius: 0.75rem;
  background: #ffffff;
  box-shadow: 0 0.25rem 0.8rem rgba(0, 0, 0, 0.12);
}

.cartao img {
  display: block;
  width: 100%;
  height: auto;
}

.cartao figcaption {
  padding: 0.8rem;
  font-weight: 700;
}

@media (max-width: 768px) {
  .galeria {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
```

```

}
}

@media (max-width: 480px) {
  .galeria {
    grid-template-columns: 1fr;
  }
}

```

Teste de responsividade

1. Abra o arquivo index.html no navegador para uma primeira conferência.
2. Pressione F12 e ative o modo de visualização para dispositivos móveis.
3. Teste larguras maiores que 768 px: a galeria deve exibir três colunas.
4. Teste entre 481 px e 768 px: a galeria deve exibir duas colunas.
5. Teste 480 px ou menos: a galeria deve exibir uma coluna.

Registro 1 - Resultado do layout

Largura testada	Colunas observadas	Funcionou?
Acima de 768 px	02 Colunas	Sim
481 a 768 px	02 Colunas	Sim
Até 480 px	01 Coluna	Sim

Etapa 4 - Executar em um servidor local

Abra o terminal na pasta do projeto e execute:

```
python -m http.server 8000
```

No navegador, acesse:

```
http://127.0.0.1:8000
```

Como encerrar

Volte ao terminal e pressione Ctrl + C. O Live Server do Visual Studio Code também pode ser utilizado como alternativa.

Etapa 5 - Minificar o CSS

A atividade utilizará um script local para evitar dependência de instalação de pacotes no laboratório. Crie o arquivo minificar_css.py com o código abaixo:

```

from pathlib import Path
import re

entrada = Path("estilos.css")
saida = Path("estilos.min.css")

css = entrada.read_text(encoding="utf-8")
css = re.sub(r"/\.*?\/", "", css, flags=re.S)
css = re.sub(r"\s+", " ", css)
css = re.sub(r"\s*([{};,:])\s*", r"\1", css)
css = css.replace(";", ";").strip()

saida.write_text(css, encoding="utf-8")
print(f"Arquivo gerado: {saida}")
print(f"Tamanho original: {entrada.stat().st_size} bytes")
print(f"Tamanho minificado: {saida.stat().st_size} bytes")

```

No terminal, execute:

```
python minificar_css.py
```

- O comando deverá criar o arquivo estilos.min.css.
- Compare os tamanhos exibidos no terminal.
- Não apague estilos.css: ele continua sendo a versão organizada para edição.

Vincular o arquivo minificado

No index.html, troque apenas a linha do link:

```
<link rel="stylesheet" href="estilos.min.css">
```

Atualize a página. O resultado visual deve permanecer igual. Se a página perder os estilos, verifique o nome do arquivo e execute novamente o script.

Registro 2 - Comparação dos arquivos

Arquivo	Tamanho em bytes	Observação
estilos.css	856 bytes	Versão organizada
estilos.min.css	625 bytes	Versão reduzida

Redução observada: _____

Opção complementar com NPM

Em laboratórios com internet e permissão para instalar pacotes, o professor pode demonstrar PostCSS e cssnano. Essa opção não é necessária para concluir a atividade e pode falhar quando o registro NPM estiver bloqueado.

```
npm init -y
npm install --save-dev postcss postcss-cli cssnano
npx postcss estilos.css -o estilos.min.css
```

Etapa 6 - Analisar com Google Lighthouse

1. Mantenha a página aberta pelo endereço do servidor local.
2. Pressione F12 para abrir as Ferramentas do Desenvolvedor.
3. Abra a guia Lighthouse. Se ela não estiver visível, procure no menu de guias adicionais.
4. Selecione Performance, Acessibilidade e Boas Práticas.
5. Escolha a análise para computador ou dispositivo móvel.
6. Clique para analisar o carregamento da página.
7. Aguarde o relatório e registre a nota e as principais recomendações.

Interpretação correta

O Lighthouse apresenta uma estimativa baseada no teste realizado. A nota pode variar entre execuções. O mais importante é compreender as recomendações e verificar se elas fazem sentido para o projeto.

Registro 3 - Resultado da auditoria

Categoria	Resultado	Uma recomendação encontrada
Performance	84	Tente auditar em página anônima e sem extensões
Acessibilidade	100	
Boas Práticas	100	

Etapa 7 - Perguntas para reflexão

1. Como o CSS Grid ajudou a adaptar a galeria para telas diferentes?

Ajudou a adaptar galerias de fotos de forma automática e limpa ao usar colunas fluidas, repetição inteligente e ajuste de tamanho sem código complexo.

2. Qual é a função do atributo `loading="lazy"`?

Ele só carrega o essencial do site para funcionar, deixando ele leve e rápido.

3. O resultado visual mudou após a minificação? Explique.

Não identifiquei nenhuma mudança após a minificação.

4. Qual foi a diferença de tamanho entre `estilos.css` e `estilos.min.css`?

Teve uma diferença de 231 bytes.

5. Qual recomendação do Lighthouse parece mais importante para este projeto? Por quê?

Tentar auditar em página anônima e sem extensões, para melhorar o desempenho do site.

6. Por que performance e acessibilidade devem ser consideradas durante o desenvolvimento, e não apenas no final?

Para que seja acessível para todos e sem travamentos ou demora quando é acessado o site.

Checklist de entrega

- ☐ index.html abre sem erros e utiliza meta viewport.
- ☐ A página possui três cartões com imagens e legendas.
- ☐ Todas as imagens têm alt, width, height e loading="lazy".
- ☐ estilos.css utiliza CSS Grid, gap e Media Queries.
- ☐ A galeria apresenta 3, 2 e 1 colunas conforme a largura.
- ☐ minificar_css.py gera estilos.min.css.
- ☐ O HTML foi atualizado para usar estilos.min.css.
- ☐ A página foi aberta em servidor local.
- ☐ O Lighthouse foi executado e os resultados foram registrados.
- ☐ As perguntas de reflexão foram respondidas.

CrITÉRIOS de avaliação

Critério	Evidência esperada	Pontos
Estrutura HTML	Tags organizadas, link correto e conteúdo semântico.	2,0
CSS Grid e responsividade	Grade funcional em três faixas de largura.	2,5
Imagens e acessibilidade	alt, dimensões e Lazy Loading configurados.	1,5
Minificação	Arquivo minificado gerado e vinculado corretamente.	1,5
Análise de performance	Lighthouse executado e recomendações registradas.	1,5
Reflexão e organização	Respostas coerentes e pasta organizada.	1,0
TOTAL		10,0

Solução de problemas

Problema	O que verificar
As imagens não aparecem.	Confira nomes, extensões e se os arquivos estão na mesma pasta.
O CSS não foi aplicado.	Confira o href do link e o nome estilos.css ou estilos.min.css.
A grade não muda no celular.	Confira meta viewport, Media Queries e fechamento das chaves.
python não é reconhecido.	Tente py -m http.server 8000 no Windows ou solicite apoio do professor.
O arquivo minificado não foi criado.	Abra o terminal na pasta correta e confira o nome estilos.css.
A guia Lighthouse não aparece.	Abra o menu de guias adicionais das DevTools ou atualize o Chrome.

Referência da atividade

Adaptada de [SIS]ANO2C3B3S15A3AP - Otimizando a performance de uma aplicação web com Lazy Loading, minificação e análise de desempenho.